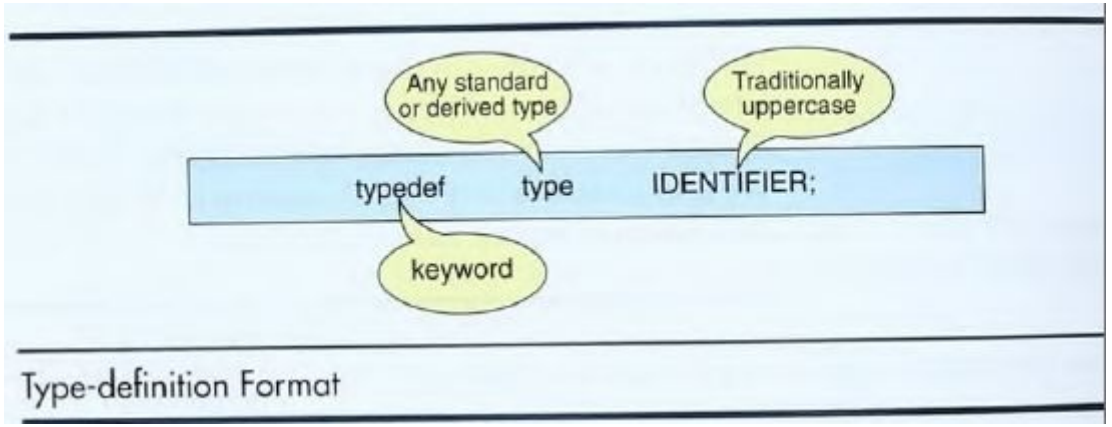
Typedef

- The typedef is a keyword that is used to provide existing data types with a new name.
- The C typedef keyword is used to redefine the name of already existing data types.
- When names of datatypes become difficult to use in programs, typedef is used with user-defined datatypes, which behave similarly to defining an alias for commands.




```
typedef int INTEGER;
```

Typedef

```
#include <stdio.h>

typedef int Integer;

int main() {

    // n is of type int, but we are using
      // alias Integer
    Integer n = 10;

    printf("%d", n);
    return 0;
}
```

Typedef

```
char * StringPtrAry[20];
```

Instead:

```
typedef char * STRING;
STRING StringPtrAry[20];
```

Enumerated Data Types:

Enumerated data type is a user defined type. It is mainly used to assign names to integral constants, the names make a program easy to read and maintain.

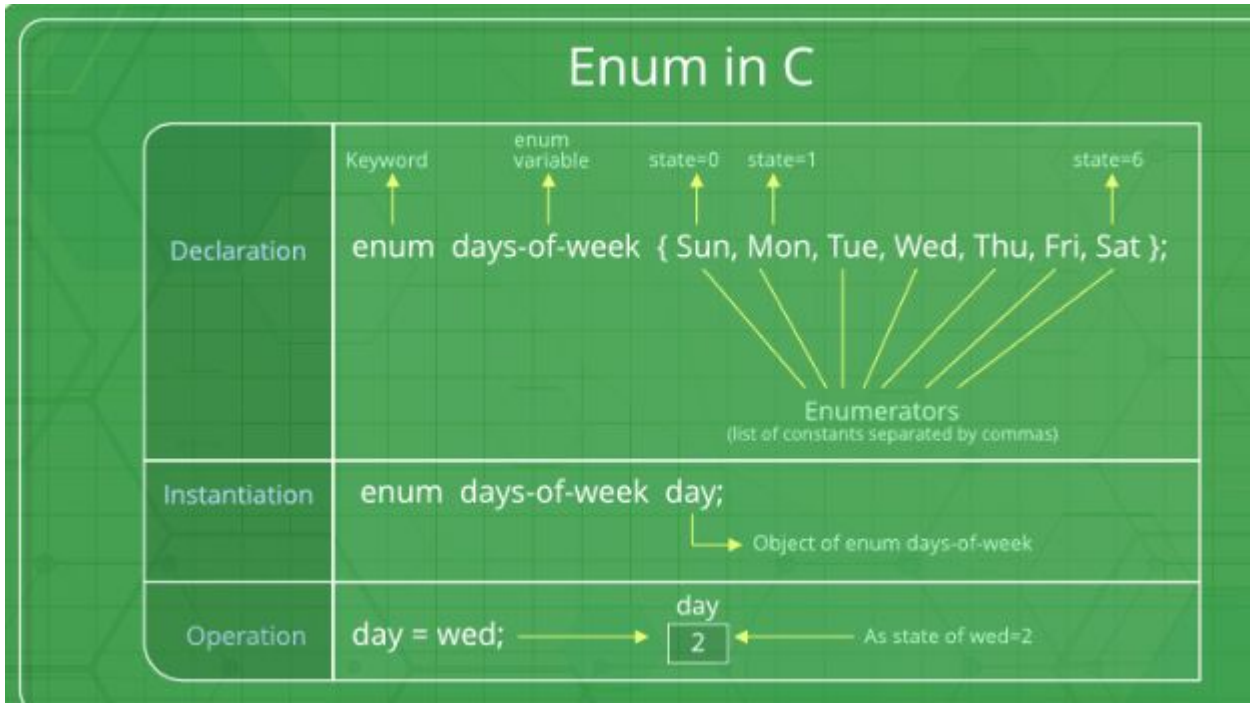
Declaring an Enumerated Type:

To declare an enumerated type, we must declare its identifier and its values. Because it is derived from integer type, its operations are the same as for integers.

```
enum typeName {identifier list};
```

The keyword enum followed by identifier and a set of enumeration constants enclosed in set of braces. The statement is terminated with a semicolon. The enumeration identifiers are also known as enumeration list.

An **enumerated data type (or enum)** is a data type that consists of a set of named values called **enumerators** or **members**. These values are used to represent a collection of related constants, making the code more readable, maintainable, and less prone to errors.



Enumerated Data Types:

```
// In both of the below cases, "day" is
// defined as the variable of type week.

enum week{Mon, Tue, Wed};
enum week day;

// Or

enum week{Mon, Tue, Wed}day;
```

Enumerated Data Types:

```
// An example program to demonstrate working
// of enum in C
#include<stdio.h>

enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};

int main()
{
    enum week day;
    day = Wed;
    printf("%d",day);
    return 0;
}
```

Enumerated Data Types:

```
#include <stdio.h>
enum State {Working = 1, Failed = 0, Freezed = 0};

int main()
{
   printf("%d, %d, %d", Working, Failed, Freezed);
   return 0;
}
```

Enumerated Data Types Advantages:

Readability: By using meaningful names instead of raw integers, the code becomes easier to understand.

Maintainability: If you need to change a value (e.g., the value of Sunday), you can do so in one place, making it easier to maintain the code.

Structures

A structure is a user defined data type that groups logically related data items of different data types into a single unit. All the elements of a structure are stored at contiguous memory locations. A variable of structure type can store multiple data items of different data types under the one name. As the data of employee in company that is name, Employee ID, salary, address, phone number is stored in structure data type.

```
struct <struct_name>
{
<data_type> <variable_name>;
<data_type> <variable_name>;
.....
<data_type> <variable_name>;
};
```

Creating a structure

You can create a structure by using the struct keyword and declare each of its members inside curly braces:

```
struct MyStructure {   // Structure declaration
  int myNum;           // Member (int variable)
  char myLetter;       // Member (char variable)
}; // End the structure with a semicolon
```

Accessing Structure

Use the struct keyword inside the main() method, followed by the name of the structure and then the name of the structure variable:

```
struct myStructure {
  int myNum;
  char myLetter;
};

int main() {
  struct myStructure s1;
  return 0;
}
```

Accessing structure members

To access members of a structure, use the dot syntax (.):

```
struct myStructure {
  int myNum;
  char myLetter;
};

int main() {
    struct myStructure s1;

  s1.myNum = 13;
  s1.myLetter = 'B';

    printf("My number: %d\n", s1.myNum);
  printf("My letter: %c\n", s1.myLetter);

  return 0;
}
```

Output:-

```
My number: 13
My letter: B
```

Strings in structures in C

```c
struct myStructure {
  int myNum;
  char myLetter;
  char myString[30];  // String
};

int main() {
  struct myStructure s1;

  // Trying to assign a value to the string
  s1.myString = "Some text";

  // Trying to print the value
  printf("My string: %s", s1.myString);

  return 0;
}
```

Output:-

An error will occur, so use strcpy to assign the values to s1.myString

Using strcpy( ) to assign the values to structure members

```c
struct myStructure {
  int myNum;
  char myLetter;
  char myString[30]; // String
};

int main() {
  struct myStructure s1;

  // Assign a value to the string using the strcpy function
  strcpy(s1.myString, "Some text");

  // Print the value
  printf("My string: %s", s1.myString);

  return 0;
}
```

OUTPUT:-

```
My string: Some text
```

Assigning the values to structure members

```c
// Create a structure
struct myStructure {
  int myNum;
  char myLetter;
  char myString[30];
};

int main() {
  // Create a structure variable and assign values to it
  struct myStructure s1 = {13, 'B', "Some text"};

  // Print values
  printf("%d %c %s", s1.myNum, s1.myLetter, s1.myString);

  return 0;
}
```

Initialize Structure Members

Structure members cannot be initialized with the declaration. For example, the following C program fails in the compilation.

```
struct Point
{
    int x = 0;  // COMPILER ERROR:  cannot initialize members here
    int y = 0;  // COMPILER ERROR:  cannot initialize members here
};
```

Structure Initialization

- Using Assignment Operator.

```
struct structure_name str;
str.member1 = value1;
str.member2 = value2;
str.member3 = value3;
.
```

- Using Initializer List.

```
struct structure_name str = { value1, value2, value3 };
```

- Using Designated Initializer List.

```
struct structure_name str = { .member1 = value1, .member2 = value2, .member3 = value3 };
```

This feature is added in c99 standard.

Array within a structure

- A structure is a data type in C that allows a group of related variables to be treated as a single unit instead of separate entities.
- A structure may contain elements of different data types – int, char, float, double, etc. It may also contain an array as its member.
- Such an array is called an array within a structure.
- An array within a structure is a member of the structure and can be accessed just as we access other elements of the structure.

Arrays within a structure

```
#include <stdio.h>
struct candidate {
        int roll_no;
        char grade;
        float marks[4];
};
void display(struct candidate a1)
{
        printf("Roll number : %d\n", a1.roll_no);
        printf("Grade : %c\n", a1.grade);
        printf("Marks secured:\n");
        int i;
        int len = sizeof(a1.marks) / sizeof(float);
        for (i = 0; i < len; i++) {
        printf("Subject %d : %.2f\n", i + 1, a1.marks[i]);
        }
}
```

```
int main()
{
        struct candidate A = { 1, 'A', { 98.5, 77,
89, 78.5 } };
        display(A);
        return 0;
}
```

OUTPUT:-

```
Roll number : 1
Grade : A
Marks secured:
Subject 1 : 98.50
Subject 2 : 77.00
Subject 3 : 89.00
Subject 4 : 78.50
```

Array of Structures

- An array is a collection of data items of the same type.
- Each element of the array can be int, char, float, double, or even a structure.
- We have seen that a structure allows elements of different data types to be grouped together under a single name.
- This structure can then be thought of as a new data type in itself.
- So, an array can comprise elements of this new data type.
- An array of structures finds its applications in grouping the records together and provides for fast access.

Array of Structures--Example

```
#include <stdio.h>
struct class {
        int roll_no;
        char grade;
        float marks;
};
void display(struct class class_record[3])
{
int i, len = 3;
for (i = 0; i < len; i++) {
printf("Roll number : %d\n", class_record[i].roll_no);
printf("Grade : %c\n", class_record[i].grade);
printf("Average marks : %.2f\n",class_record[i].marks);
printf("\n");
}
}
int main()
{
struct class class_record[3] = { { 1, 'A', 89.5f },{ 2, 'C', 67.5f },{ 3, 'B', 70.5f } };
display(class_record);
}
```

OUTPUT:-

```
Roll number : 1
Grade : A
Average marks : 89.50

Roll number : 2
Grade : C
Average marks : 67.50

Roll number : 3
Grade : B
Average marks : 70.50
```

UNIONS

- The Union is a user-defined data type in C language that can contain elements of the different data types just like structure.
- But unlike structures, all the members in the C union are stored in the same memory location.
- Due to this, only one member can store data at the given instance.

Union Declaration

```
union union_name {
 datatype member1;
  datatype member2;
};
```

UNIONS

1. Defining Union Variable with Declaration

```
union union_name {
    datatype member1;
    datatype member2;
    ...
} var1, var2, ...;
```

2. Defining Union Variable after Declaration

union union_name var1, var2, var3...;
where union_name is the name of an already declared union.

UNIONS

```
#include <stdio.h>

union un {
        int member1;
        char member2;
        float member3;
};
int main()
{
        union un var1;
        var1.member1 = 15;
        printf("The value stored in member1 = %d", var1.member1);

        return 0;
}
```

OUTPUT:-

```
The value stored in member1 = 15
```

Size of Union

The size of the union will always be equal to the size of the largest member of the array. All the less-sized elements can store the data in the same space without any overflow.

```
#include <stdio.h>
union test1 {
        int x;
        int y;
} Test1;

union test2 {
        int x;
        char y;
} Test2;

union test3 {
        int arr[10];
        char y;
} Test3;
```

```
int main()
{

int size1 =sizeof(Test1);
int size2 = sizeof(Test2);
int size3 = sizeof(Test3);

printf("Sizeof test1: %d\n", size1);
printf("Sizeof test2: %d\n", size2);
printf("Sizeof test3: %d", size3);
return 0;
}
```

Output:-

```
Sizeof test1: 4
Sizeof test2: 4
Sizeof test3: 40
```

Differentiate between Structures and Unions

Structures	Unions
The size of the structure is equal to or greater than the total size of all of its members.	The size of the union is the size of its largest member.
The structure can contain data in multiple members at the same time.	Only one member can contain data at the same time.
It is declared using the struct keyword.	It is declared using the union keyword.